

Tugas 12: Praktikum Mandiri 12

Fatih Dzakwan Susilo - 0110224235

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

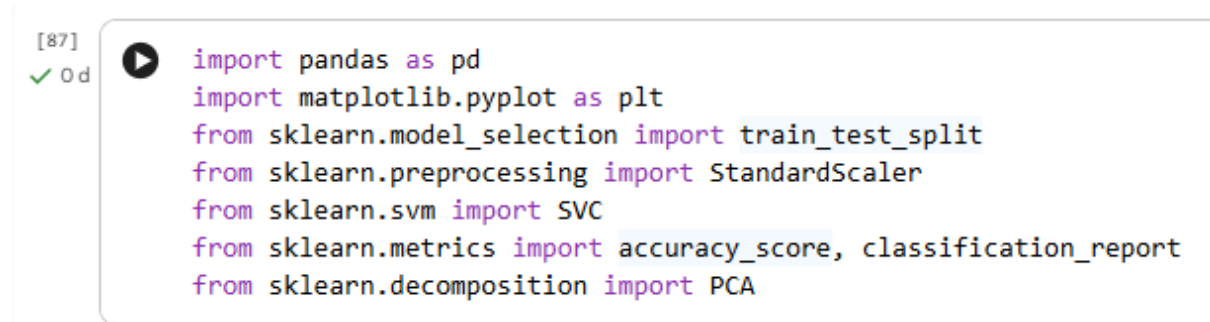
E-mail: fatihdzakwansusilo@gmail.com

1. Praktikum Mandiri

Link Github : https://github.com/FatihDzkwn/Project_Machine_Learning.git

Link Dataset : <https://www.kaggle.com/datasets/mysarahmadbhat/lung-cancer>

1.1 Import Library



```
[87]
✓ 0 d  ▶ import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA
```

Gambar 1. Import Library.

Kode ini merupakan tahap import library untuk proyek machine learning klasifikasi menggunakan Support Vector Machine (SVM). Library yang diimpor mencakup pandas untuk manipulasi data, matplotlib untuk visualisasi, serta berbagai modul dari scikit-learn untuk preprocessing, pemodelan, dan evaluasi. Secara khusus, kode ini mempersiapkan tools untuk membagi dataset, normalisasi fitur dengan StandardScaler, membangun model SVM, mengukur akurasi, dan melakukan reduksi dimensi dengan PCA. Ini adalah fondasi awal sebelum melakukan analisis dan pemodelan data lebih lanjut.

1.2 Menghubungkan Google Colab dengan Google Drive



```
[217]
✓ 2 d  from google.colab import drive
drive.mount('/content/drive')
```

▼ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Gambar 2. Menghubungkan Google Colab dengan Google Drive.

Kode ini berfungsi untuk menghubungkan atau mounting Google Drive ke dalam environment Google Colab agar dapat mengakses file-file yang tersimpan di Google Drive. Setelah menjalankan kode ini, pengguna akan diminta untuk melakukan autentikasi dengan akun Google mereka, dan setelah berhasil, Google Drive akan terpasang pada direktori '/content/drive'. Dengan demikian, semua file dan folder yang ada di Google Drive dapat diakses langsung dari notebook Colab untuk keperluan membaca dataset, menyimpan hasil analisis, atau operasi file lainnya tanpa perlu mengupload file secara manual setiap kali menjalankan notebook.

1.3 Loading dan eksplorasi data awal

```
[99] path = '/content/drive/MyDrive/Project_Machine_Learning/Praktikum_Mandiri_12/Data/'
✓ 0 d df = pd.read_csv(path + 'survey_lung_cancer.csv')
df.head()
```

	GENDER	AGE	SMOKING	YELLOW_FINGERS	ANXIETY	PEER_PRESSURE	CHRONIC_DISEASE	FATIGUE	ALLERGY	WHEEZING	ALCOHOL_CONSUMING	COUGHING	SHORTNESS OF BREATH	SWALLOWING DIFFICULTY	CHEST PAIN	LUNG_CANCER
0	M	69	1	2	2	1	1	2	1	2	2	2	2	2	2	YES
1	M	74	2	1	1	1	2	2	2	1	1	1	2	2	2	YES
2	F	59	1	1	1	2	1	2	1	2	1	2	2	1	2	NO
3	M	63	2	2	2	1	1	1	1	1	2	1	1	2	2	NO
4	F	63	1	2	1	1	1	1	1	2	1	2	2	1	1	NO

Gambar 3. Loading dan eksplorasi data awal.

Kode ini merupakan tahap loading dan eksplorasi data awal dari proyek machine learning. Pertama, variabel path didefinisikan untuk menyimpan lokasi direktori file dataset yang berada di Google Drive. Kemudian, dataset dengan nama "survey_lung_cancer.csv" dibaca menggunakan pandas dan disimpan dalam dataframe bernama df. Fungsi head() digunakan untuk menampilkan lima baris pertama dari dataset agar dapat melihat struktur data, nama kolom, dan contoh nilai yang terkandung di dalamnya sebagai langkah awal pemahaman data.

1.4 Inspeksi informasi dataset

```
[90] df.info()
✓ 0 d
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 309 entries, 0 to 308
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                ---
0   GENDER                                309 non-null    object
1   AGE                                  309 non-null    int64
2   SMOKING                              309 non-null    int64
3   YELLOW_FINGERS                       309 non-null    int64
4   ANXIETY                              309 non-null    int64
5   PEER_PRESSURE                        309 non-null    int64
6   CHRONIC_DISEASE                      309 non-null    int64
7   FATIGUE                              309 non-null    int64
8   ALLERGY                              309 non-null    int64
9   WHEEZING                             309 non-null    int64
10  ALCOHOL_CONSUMING                    309 non-null    int64
11  COUGHING                             309 non-null    int64
12  SHORTNESS OF BREATH                  309 non-null    int64
13  SWALLOWING DIFFICULTY                309 non-null    int64
14  CHEST PAIN                           309 non-null    int64
15  LUNG_CANCER                          309 non-null    object
dtypes: int64(14), object(2)
memory usage: 38.8+ KB
```

Gambar 4. Inspeksi informasi dataset.

Kode ini merupakan tahap inspeksi informasi dataset untuk memahami struktur dan karakteristik data secara menyeluruh. Fungsi info() menampilkan ringkasan penting seperti jumlah baris dan kolom, nama setiap kolom beserta tipe datanya, jumlah nilai non-null pada tiap

kolom, serta penggunaan memori oleh dataframe. Informasi ini sangat berguna untuk mengidentifikasi potensi missing values, mengetahui apakah tipe data sudah sesuai untuk analisis, dan merencanakan langkah preprocessing yang diperlukan sebelum melakukan pemodelan.

1.5 Deteksi missing values



The screenshot shows a Jupyter Notebook interface. On the left, the input code is `df.isnull().sum()` with a green checkmark and the output index `[91]`. The main area displays the output as a table with 15 rows and 2 columns. The first column lists various health-related features, and the second column shows the count of missing values for each feature, all of which are 0. Below the table, the data type is specified as `dtype: int64`.

GENDER	0
AGE	0
SMOKING	0
YELLOW_FINGERS	0
ANXIETY	0
PEER_PRESSURE	0
CHRONIC_DISEASE	0
FATIGUE	0
ALLERGY	0
WHEEZING	0
ALCOHOL_CONSUMING	0
COUGHING	0
SHORTNESS_OF_BREATH	0
SWALLOWING_DIFFICULTY	0
CHEST_PAIN	0
LUNG_CANCER	0

dtype: int64

Gambar 5. Deteksi missing values.

Kode ini merupakan tahap deteksi missing values untuk mengidentifikasi kelengkapan data dalam dataset. Fungsi `isnull()` mengecek setiap sel dalam dataframe dan mengembalikan nilai `True` jika sel tersebut kosong atau bernilai `null`, kemudian `sum()` menghitung total nilai `null` pada setiap kolom. Hasil dari kode ini menampilkan jumlah data yang hilang per kolom, sehingga dapat

diketahui apakah dataset memerlukan penanganan khusus seperti imputasi atau penghapusan baris/kolom sebelum dilakukan analisis lebih lanjut.

1.6 Deteksi dan penghapusan data duplikat

```
[92] ✓ 0 d
df.duplicated().sum()
np.int64(33)

[93] ✓ 0 d
df[df.duplicated(keep=False)]
```

	GENDER	AGE	SMOKING	YELLOW_FINGERS	ANXIETY	PEER_PRESSURE	CHRONIC_DISEASE	FATIGUE	ALLERGY	WHEEZING	ALCOHOL_CONSUMING	COUGHING	SHORTNESS_OF_BREATH	SHALLOWING_DIFFICULTY	CHEST_PAIN	LUNG_CANCER
7	F	51	2	2	2	2	1	2	2	1	1	1	2	2	1	YES
13	M	58	2	1	1	1	1	2	2	2	2	2	2	1	2	YES
23	M	60	2	1	1	1	1	2	2	2	2	2	2	1	2	YES
51	M	63	2	2	2	1	2	2	2	2	1	1	2	1	1	YES
75	M	58	2	1	1	1	1	1	2	2	2	2	1	1	1	YES
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
304	F	56	1	1	1	2	2	2	1	1	2	2	2	2	1	YES
305	M	70	2	1	1	1	1	2	2	2	2	2	2	1	2	YES
306	M	58	2	1	1	1	1	1	2	2	2	2	1	1	2	YES
307	M	67	2	1	2	1	1	2	2	1	2	2	2	1	2	YES
308	M	62	1	1	1	2	1	2	2	2	2	1	1	2	1	YES

```
66 rows x 16 columns

[94] ✓ 0 d
df = df[df.duplicated() == False]

[95] ✓ 0 d
df.duplicated().sum()
np.int64(0)
```

Gambar 6. Deteksi dan penghapusan data duplikat.

Kode ini merupakan tahap deteksi dan penghapusan data duplikat untuk memastikan kualitas dataset. Baris pertama menghitung jumlah total baris yang duplikat dalam dataset, kemudian baris kedua menampilkan semua baris duplikat beserta kembarannya menggunakan parameter `keep=False`. Setelah itu, dilakukan penghapusan baris duplikat dengan menyimpan hanya baris unik ke dalam dataframe `df`, dan baris terakhir melakukan verifikasi ulang untuk memastikan tidak ada lagi data duplikat yang tersisa setelah proses pembersihan.

1.7 Analisis statistik deskriptif

```
[96] ✓ 0 d
df.describe()
```

	AGE	SMOKING	YELLOW_FINGERS	ANXIETY	PEER_PRESSURE	CHRONIC_DISEASE	FATIGUE	ALLERGY	WHEEZING	ALCOHOL_CONSUMING	COUGHING	SHORTNESS_OF_BREATH	SHALLOWING_DIFFICULTY	CHEST_PAIN	LUNG_CANCER
count	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000	276.000000
mean	62.909420	1.543478	1.576087	1.496377	1.507246	1.521739	1.663043	1.547101	1.547101	1.550725	1.576087	1.630435	1.467391	1.557971	
std	8.379355	0.499011	0.495075	0.500895	0.500895	0.500435	0.473529	0.498681	0.498681	0.498324	0.495075	0.483564	0.499842	0.497530	
min	21.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	
25%	57.750000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	
50%	62.500000	2.000000	2.000000	1.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	1.000000	2.000000	
75%	69.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	
max	87.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	

Gambar 7. Analisis statistik deskriptif.

Kode ini merupakan tahap analisis statistik deskriptif untuk memahami distribusi dan karakteristik numerik dari dataset. Fungsi `describe()` menghasilkan ringkasan statistik yang mencakup jumlah data (`count`), nilai rata-rata (`mean`), standar deviasi (`std`), nilai minimum, kuartil pertama (25%), median (50%), kuartil ketiga (75%), dan nilai maksimum untuk setiap kolom numerik. Informasi ini sangat berguna untuk mendeteksi outlier, memahami rentang nilai data, serta melihat variabilitas dan tendensi sentral dari setiap fitur sebelum melakukan pemodelan machine learning.

1.8 Analisis distribusi kelas target

```
[97]
✓ Od # Distribusi kelas target
print("Nama Kelas:", df['LUNG_CANCER'].unique())
df['LUNG_CANCER'].value_counts()

Nama Kelas: ['YES' 'NO']

count

LUNG_CANCER
YES      238
NO       38

dtype: int64
```

Gambar 8. Analisis distribusi kelas target.

Kode ini merupakan tahap analisis distribusi kelas target untuk memahami keseimbangan kategori pada variabel dependen. Baris pertama menampilkan semua nilai unik yang terdapat dalam kolom LUNG_CANCER untuk mengetahui kategori kelas yang ada dalam dataset, seperti YES atau NO. Kemudian fungsi value_counts() menghitung dan menampilkan frekuensi kemunculan setiap kelas, sehingga dapat diketahui apakah dataset mengalami imbalanced class atau sudah seimbang, yang penting untuk menentukan strategi pemodelan dan evaluasi yang tepat.

1.9 Preprocessing dan pemisahan fitur-label

```
[98]
✓ Od # Fitur dan label
# X = wine.data      # bisa juga df[wine.feature_names].values
# y = wine.target    # df['target'].values

# Menggunakan DataFrame 'df' yang sudah dimuat
# Mengubah kolom 'GENDER' dan 'LUNG_CANCER' menjadi numerik (Label Encoding)
df_encoded = df.copy()
df_encoded['GENDER'] = df_encoded['GENDER'].map({'M': 0, 'F': 1})
df_encoded['LUNG_CANCER'] = df_encoded['LUNG_CANCER'].map({'NO': 0, 'YES': 1})

X = df_encoded.drop('LUNG_CANCER', axis=1) # Menjadikan semua kolom kecuali 'LUNG_CANCER' sebagai fitur
y = df_encoded['LUNG_CANCER']              # Menjadikan 'LUNG_CANCER' sebagai label

print("Shape X:", X.shape)
print("Shape Y:", y.shape)

... Shape X: (276, 15)
Shape Y: (276,)
```

Gambar 9. Preprocessing dan pemisahan fitur-label.

Kode ini merupakan tahap preprocessing dan pemisahan fitur-label untuk mempersiapkan data sebelum pemodelan. Pertama, dilakukan label encoding manual pada kolom kategorikal GENDER dengan mengubah M menjadi 0 dan F menjadi 1, serta kolom LUNG_CANCER dengan mengubah NO menjadi 0 dan YES menjadi 1 agar dapat diproses oleh algoritma machine learning. Setelah encoding, data dipisahkan menjadi fitur X yang berisi semua kolom kecuali LUNG_CANCER dan label y yang berisi kolom LUNG_CANCER sebagai target prediksi. Terakhir, shape dari X dan y

ditampilkan untuk memverifikasi dimensi data, dimana X menunjukkan jumlah sampel dan fitur, sedangkan y menunjukkan jumlah sampel label.

1.10 Standarisasi fitur

```
[100]
✓ Od # Standarisasi (mean=0, std=1)
    scaler = StandardScaler()

    X_train_scaled = scaler.fit_transform(X_train) # fit + transform pada data train
    X_test_scaled = scaler.transform(X_test) # hanya transform pada data test

    X_train_scaled[:5]
```

```
... array([[ 1.05611771,  0.83101961, -1.12607337,  0.81649658, -0.98198051,
            1.03704952,  0.94686415,  0.71919495,  0.92973479, -1.06579657,
            0.92973479,  0.83989663, -1.39044357, -0.93826536, -1.14707867],
          [ 1.05611771,  1.3336294 ,  0.8880416 ,  0.81649658,  1.01835015,
            1.03704952, -1.05611771,  0.71919495, -1.07557554,  0.93826536,
            -1.07557554,  0.83989663,  0.71919495,  1.06579657, -1.14707867],
          [ 1.05611771, -0.29985244, -1.12607337, -1.22474487, -0.98198051,
            -0.96427411,  0.94686415,  0.71919495, -1.07557554, -1.06579657,
            -1.07557554, -1.1906227 ,  0.71919495, -0.93826536, -1.14707867],
          [ 1.05611771, -0.29985244,  0.8880416 ,  0.81649658,  1.01835015,
            1.03704952, -1.05611771,  0.71919495,  0.92973479, -1.06579657,
            -1.07557554, -1.1906227 ,  0.71919495,  1.06579657,  0.87177979],
          [ 1.05611771,  0.20275736,  0.8880416 ,  0.81649658,  1.01835015,
            1.03704952, -1.05611771,  0.71919495, -1.07557554,  0.93826536,
            -1.07557554,  0.83989663,  0.71919495,  1.06579657, -1.14707867]])
```

Gambar 10. Standarisasi fitur.

Kode ini merupakan tahap standarisasi fitur untuk menormalisasi skala data agar memiliki mean 0 dan standar deviasi 1. Objek `StandardScaler` dibuat terlebih dahulu, kemudian metode `fit_transform` diterapkan pada data training untuk mempelajari parameter statistik seperti mean dan standar deviasi sekaligus melakukan transformasi. Pada data testing hanya dilakukan transform tanpa fit untuk menghindari data leakage, menggunakan parameter yang sudah dipelajari dari data training. Baris terakhir menampilkan lima sampel pertama dari data training yang sudah distandarisasi untuk memverifikasi hasil transformasi, dimana nilai-nilai fitur sekarang berada dalam skala yang sebanding dan berpusat di sekitar nol.

1.11 Pemodelan dan evaluasi SVM tanpa reduksi dimensi (PCA)

```
[101]
✓ Od # Model SVM tanpa PCA
svm_no_pca = SVC(kernel='linear', gamma = "scale", random_state=42)
svm_no_pca.fit(X_train_scaled, y_train)

# Prediksi dan evaluasi
y_pred_no_pca = svm_no_pca.predict(X_test_scaled)

acc_no_pca = accuracy_score(y_test, y_pred_no_pca)
print("Akurasi tanpa PCA:", acc_no_pca)

print("\nClassification Report (tanpa PCA):")
print(classification_report(y_test, y_pred_no_pca, target_names=['NO', 'YES']))
```

✓ Akurasi tanpa PCA: 0.9285714285714286

Classification Report (tanpa PCA):

	precision	recall	f1-score	support
NO	0.70	0.88	0.78	8
YES	0.98	0.94	0.96	48
accuracy			0.93	56
macro avg	0.84	0.91	0.87	56
weighted avg	0.94	0.93	0.93	56

Gambar 11. Pemodelan dan evaluasi SVM tanpa reduksi dimensi (PCA).

Kode ini merupakan **tahap pemodelan dan evaluasi SVM tanpa reduksi dimensi** untuk mengukur performa model pada data asli. Model SVM dengan kernel linear dibuat dan dilatih menggunakan data training yang sudah distandarisasi, dengan parameter gamma diatur sebagai "scale" dan random_state untuk reproduibilitas hasil. Setelah pelatihan, model digunakan untuk memprediksi label pada data testing, kemudian akurasi dihitung dengan membandingkan prediksi terhadap label sebenarnya. Terakhir, classification report ditampilkan untuk memberikan evaluasi mendetail mencakup precision, recall, dan f1-score untuk setiap kelas NO dan YES, sehingga dapat memahami performa model secara komprehensif pada kedua kategori kanker paru-paru.